

Ótimo! Ter créditos nas duas plataformas é o cenário ideal para a arquitetura que estamos montando. Isso te dá o controle absoluto: você usa a TypeSafe como sua "porta de entrada" (System 1) e o OpenRouter como seu "motor de execução" (System 2).

Como você vai construir um *middleware* (o proxy local) para interceptar o Claude Code, o fluxo do seu dinheiro e das suas chaves vai funcionar assim:

1. **Os créditos do Jev (TypeSafe):** Vão pagar apenas as frações de centavos para a camada de decisão rápida (avaliar a intenção do seu prompt).
2. **Os créditos do OpenRouter:** Vão pagar a geração de texto pesada (o código e a resposta final), permitindo que você alterne entre o Claude 3.5 Sonnet, Llama, Gemini, etc.

Aqui está o guia definitivo de como plugar essas duas chaves no seu proxy local.

Passo 1: Obter as Chaves

- Vá no console.typesafe.ai e gere sua chave (vamos chamá-la de JEV_API_KEY).
- Vá no openrouter.ai/workspaces, entre no seu workspace e gere sua chave (vamos chamá-la de OPENROUTER_API_KEY).

Passo 2: O Código do Proxy Roteador (Seu Middleware)

Crie uma pasta, rode `npm init -y`, instale o express (`npm install express`) e crie um arquivo chamado `proxy.js`.

Este código intercepta o Claude Code, usa sua chave da TypeSafe para a triagem e sua chave do OpenRouter para a resposta:

JavaScript

```
import express from 'express'
```

```
const app = express()
```

```
app.use(express.json({ limit: '10mb' }))
```

```
// Chaves que você acabou de gerar
```

```
const JEV_API_KEY = "COLE_SUA_CHAVE_TYPESAFE_AQUI"
```

```
const OPENROUTER_API_KEY = "COLE_SUA_CHAVE_OPENROUTER_AQUI"
```

```
app.post('/v1/messages' async (req, res) =>
```

```
  try
```

```
    const claudePayload = req.body
```

```
    // Extrai a última mensagem que você digitou no Claude Code
```

```
    const lastMessage = claudePayload.messages[claudePayload.messages.length
```

```
1] content
```

```
    // =====
```

```

// CAMADA 1: JEV (Consome créditos do console.typesafe.ai)
// =====
console.log("1. Avaliando prompt com Jev...")
const jevResponse = await fetch("https://api.typesafe.ai/v1/decisions" // Endpoint oficial da
TypeSafe
  method "POST"
  headers
    "Authorization" `Bearer ${JEV_API_KEY}`
    "Content-Type" "application/json"
  }
  body JSON.stringify({
    model "jev-1.13"
    state lastMessage
    questions
    task_complexity
    type "choice"
    instructions "Classifique o nível desta requisição de programação."
    criteria
      "complex" "Arquitetura, refatoração profunda, design patterns, ou debug
complexo."
      "simple" "Dúvidas simples, git, comandos de terminal, ou explicações curtas."
  })
  }
} then r => JSON()

```

```

const complexity = jevResponse.answers?.task_complexity?.choice || "complex"
console.log(`[Decisão do Jev]: Tarefa classificada como '${complexity}'`)

```

```

// =====
// CAMADA 2: OPENROUTER (Consome créditos do openrouter.ai)
// =====
// Se for complexo, gastamos com o Claude. Se for simples, usamos um modelo barato (ex:
Llama 3).
const targetModel = complexity === "complex" ? "anthropic/claude-3.5-sonnet"
"meta-llama/llama-3-8b-instruct"
console.log(`2. Roteando para: ${targetModel}`)

```

```

const openRouterResponse = await fetch("https://openrouter.ai/api/v1/chat/completions"
  method "POST"
  headers
    "Authorization" `Bearer ${OPENROUTER_API_KEY}`

```

```

    "Content-Type" "application/json"
  }
  body JSON.stringify({
    model targetModel
    messages claudePayload.messages // Repassa o histórico inteiro
    system claudePayload.system // Repassa o system prompt do Claude Code
  }) then r => json(r)

const generatedText = openRouterResponse.choices[0].message.content

// =====
// CAMADA 3: TRADUÇÃO PARA O CLAUDE CODE (Padrão Anthropic)
// =====
// O Claude Code espera a resposta no formato exato da Anthropic.
// Transformamos o retorno do OpenRouter no JSON que a CLI entende:
res.json({
  id openRouterResponse.id || `msg_${Date.now()}`
  type "message"
  role "assistant"
  content [ { type "text" text generatedText } ]
  model targetModel
  usage
  input_tokens openRouterResponse.usage?.prompt_tokens || 0
  output_tokens openRouterResponse.usage?.completion_tokens || 0
})

catch (error) {
  console.error("Erro no Proxy:", error)
  res.status(500).json({ error message "Erro interno no Proxy Jev." })
}

const PORT = 3000
app.listen(PORT, () => console.log(`🚀 Proxy Jev/OpenRouter rodando na porta ${PORT}`))

```

Passo 3: Inicializar o ecossistema

1. No seu terminal, inicie o servidor proxy que você acabou de criar:

```

Bash
node proxy.js

```

2. Abra um **novo terminal** (deixe o proxy rodando no primeiro) e injete a variável de ambiente para "enganar" o Claude Code, fazendo-o apontar para o seu proxy em vez dos servidores da Anthropic:

```
Bash
```

```
export ANTHROPIC_BASE_URL="http://localhost:3000"
```

(Se estiver usando Windows PowerShell, o comando é:

\$env:ANTHROPIC_BASE_URL="http://localhost:3000")

3. No mesmo terminal onde você definiu a variável, inicie o Claude Code:

```
Bash
```

Como testar se deu certo?

No chat do Claude Code, digite algo super simples como: *"Qual o comando git para ver o status?"*.

Olhe para o terminal onde o seu proxy.js está rodando. Você verá os *logs* confirmando que o Jev bateu na TypeSafe, classificou como simple e gastou apenas os créditos do OpenRouter referentes ao modelo mais barato (Llama) para responder na sua tela!